

Image Preprocessing Pipeline

Technical Documentation & Logic Specification

Adaptive, quality-aware enhancement of CCTV frames prior to anomaly detection, and a dedicated restoration sub-pipeline for license-plate recognition once an anomaly (violation) is flagged.

1 Overview

The Image Preprocessing Pipeline conditions raw CCTV frames before they are handed to the downstream anomaly (traffic-violation) detector, and subsequently restores a cropped license-plate region before Optical Character Recognition (OCR). The pipeline is split into two cooperating stages, illustrated in Figure 1 and 2:

- (i) **Pre-Anomaly-Detection Stage** – a *quality-gated* enhancement branch that measures how dark an incoming frame is, conditionally brightens and denoises it, and finally deblurs it before it is passed to a YOLO-based detector.
- (ii) **Post-Anomaly-Detection Stage** – once a violation has been detected and a vehicle’s number-plate region is cropped, a GAN-based restoration network (LPDGAN) repairs blur/low-light artefacts specifically in that region before OCR is run.

Unlike a fixed, always-on enhancement filter, the pre-detection stage uses an **image-quality score** to decide *whether* aggressive low-light enhancement (Zero-DCE++) is even necessary – avoiding unnecessary artefacts on frames that are already well lit. Every frame, regardless of this decision, is still deblurred via Wiener deconvolution immediately before detection, since motion blur from moving vehicles is independent of ambient illumination.

2 Stage 1: Pre-Anomaly-Detection Pipeline

2.1 Pipeline Diagram

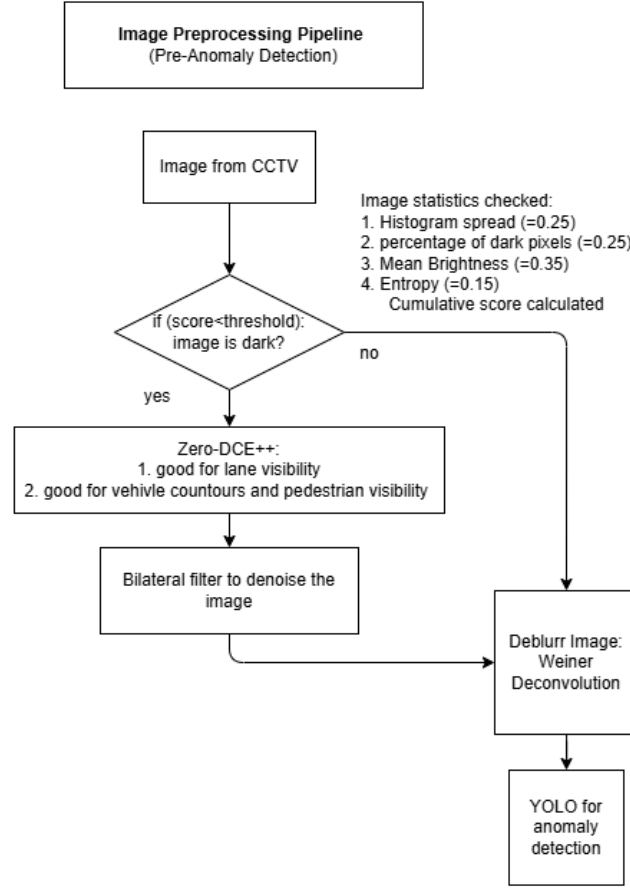


Figure 1: Pre-anomaly Image Processing

2.2 Stage Description

2.2.1 Image Acquisition

The pipeline takes the raw frame I_{raw} captured from a CCTV camera at the intersection as input. No assumption is made about illumination, weather, or motion-blur conditions at this point – all such degradations are screened for in the next step.

2.2.2 Image Quality Scoring (Darkness Gate)

Rather than always invoking a low-light enhancement network – which can introduce artefacts and unnecessary latency on frames that are already well exposed – the pipeline first computes a **cumulative darkness score** Q from four normalized statistics of I_{raw} , each weighted by an empirically tuned coefficient:

$$Q(I_{raw}) = w_1 \cdot (1 - H_{spread}) + w_2 \cdot P_{dark} + w_3 \cdot B_{mean} + w_4 \cdot E_{norm} \quad (1)$$

with weights

$$w_1 = 0.25, \quad w_2 = 0.25, \quad w_3 = 0.35, \quad w_4 = 0.15, \quad \sum_i w_i = 1.$$

Each term is defined as follows, where I_{raw} is first converted to grayscale with L intensity levels (typically $L = 256$) and N is the total pixel count:

Histogram spread H_{spread} : a normalized measure of how widely pixel intensities are distributed across the dynamic range. Letting $h(i)$ denote the (normalized) intensity histogram,

$$H_{spread} = \frac{\sigma_I}{L-1}, \quad \sigma_I = \sqrt{\sum_{i=0}^{L-1} h(i) (i - \mu_I)^2}, \quad \mu_I = \sum_{i=0}^{L-1} i h(i). \quad (2)$$

A low spread (pixels clustered in a narrow band) is characteristic of flat, dark frames.

Percentage of dark pixels P_{dark} : fraction of pixels below an intensity threshold τ_{dark} (e.g. $\tau_{dark} = 50$ on a 0–255 scale),

$$P_{dark} = \frac{1}{N} \sum_{p \in I_{raw}} \mathbb{I}[I_{raw}(p) < \tau_{dark}]. \quad (3)$$

Mean brightness B_{mean} : normalized average intensity,

$$B_{mean} = 1 - \frac{1}{N(L-1)} \sum_{p \in I_{raw}} I_{raw}(p) \quad (4)$$

(inverted so that a *darker* image yields a *higher* contribution, consistent with the other terms).

Normalized entropy E_{norm} : Shannon entropy of the intensity distribution, inverted and normalized to $[0, 1]$,

$$E_{norm} = 1 - \frac{1}{\log_2 L} \left(- \sum_{i=0}^{L-1} h(i) \log_2 h(i) \right). \quad (5)$$

Low entropy (little texture/information content) often co-occurs with under-exposure.

The frame is routed to the enhancement branch whenever the cumulative score exceeds a calibrated threshold θ :

$$\text{Decision} = \begin{cases} \text{dark (enhance)} & Q(I_{raw}) \geq \theta \\ \text{normal (skip enhancement)} & Q(I_{raw}) < \theta \end{cases} \quad (6)$$

2.2.3 Conditional Branch – Low-Light Enhancement (Zero-DCE++)

If the frame is classified as *dark*, it is passed through **Zero-DCE++** (Zero-Reference Deep Curve Estimation), a lightweight, unsupervised image-enhancement network that learns a pixel-wise

higher-order curve mapping $\mathcal{F}_\Theta$ without requiring paired low/normal-light training data. For an input pixel intensity x , the network iteratively applies a learned quadratic curve:

$$LE_n(x) = LE_{n-1}(x) + \alpha_n(x) LE_{n-1}(x) (1 - LE_{n-1}(x)), \quad n = 1, \dots, N_{iter} \quad (7)$$

where $LE_0(x) = x$ is the original pixel value, $\alpha_n(x) \in [-1, 1]$ is a pixel-wise curve parameter map predicted by a small CNN (*DCE-Net*), and N_{iter} controls the strength of enhancement. Zero-DCE++ is selected here because it:

- operates without ground-truth references, making it robust across arbitrary CCTV lighting conditions;
- is lightweight enough for near-real-time inference on edge/intersection hardware;
- empirically improves **lane-marking visibility** and **vehicle contour / pedestrian visibility** – both critical to the downstream detection task.

2.2.4 Bilateral Denoising

Brightening dark regions (via curve estimation) tends to amplify sensor noise. The enhanced frame is therefore passed through a **bilateral filter**, which smooths noise while explicitly preserving edges – important for retaining lane lines, plate edges, and vehicle boundaries. For a pixel p :

$$I_{bf}(p) = \frac{1}{W_p} \sum_{q \in \Omega(p)} I_{LE}(q) G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_{LE}(p) - I_{LE}(q)|) \quad (8)$$

$$W_p = \sum_{q \in \Omega(p)} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_{LE}(p) - I_{LE}(q)|) \quad (9)$$

where $\Omega(p)$ is a local neighborhood around p , G_{σ_s} is a spatial Gaussian kernel (controls how far neighboring pixels contribute), and G_{σ_r} is a range/intensity Gaussian kernel (controls how strongly intensity-similar pixels contribute, suppressing blending across edges).

2.2.5 Deblurring (Wiener Deconvolution)

Both branches – the enhanced-and-denoised path *and* the path for frames already classified as *normal* – converge at a single, mandatory deblurring step. Motion blur from vehicles crossing the intersection is modeled as a linear, spatially invariant degradation:

$$g(x, y) = (f * h)(x, y) + n(x, y) \quad (10)$$

where f is the latent sharp image, h is the (motion) point-spread function (PSF), n is additive noise, and $*$ denotes 2-D convolution. In the frequency domain this becomes $G(u, v) = F(u, v)H(u, v) + N(u, v)$. The **Wiener deconvolution** filter recovers an estimate $\hat{F}(u, v)$ that minimizes the mean-squared error between the estimate and the true image:

$$\hat{F}(u, v) = \left[\frac{H^*(u, v)}{|H(u, v)|^2 + \frac{S_n(u, v)}{S_f(u, v)}} \right] G(u, v) \quad (11)$$

where $H^*(u, v)$ is the complex conjugate of the PSF's Fourier transform, and $S_n(u, v)/S_f(u, v)$ is the noise-to-signal power ratio (often approximated by a single regularization constant K when per-frequency spectra are unavailable, giving the commonly implemented form $\hat{F}(u, v) = \frac{H^*}{|H|^2 + K} G(u, v)$). The deblurred image $\hat{f} = \mathcal{F}^{-1}\{\hat{F}\}$ is what is finally handed to the detector.

2.3 Algorithmic Summary – Stage 1

```

1 import cv2
2 import numpy as np
3
4 # ---- Tunable weights (Eq. 1) and threshold ----
5 W1, W2, W3, W4 = 0.25, 0.25, 0.35, 0.15 # histogram, dark-pct,
    brightness, entropy
6 THRESHOLD = 0.55 # darkness gate, tuned
    empirically
7 DARK_PIXEL_TAU = 50 # 0-255 scale
8
9 def histogram_spread(gray):
10     hist, _ = np.histogram(gray, bins=256, range=(0, 256), density=True)
11     levels = np.arange(256)
12     mu = np.sum(levels * hist)
13     sigma = np.sqrt(np.sum(hist * (levels - mu) ** 2))
14     return sigma / 255.0
15
16 def pct_dark_pixels(gray, tau=DARK_PIXEL_TAU):
17     return float(np.mean(gray < tau))
18
19 def mean_brightness_score(gray):
20     return 1.0 - (gray.mean() / 255.0)
21
22 def normalized_entropy_score(gray):
23     hist, _ = np.histogram(gray, bins=256, range=(0, 256), density=True)
24     hist = hist[hist > 0]
25     entropy = -np.sum(hist * np.log2(hist))
26     return 1.0 - (entropy / np.log2(256))
27
28 def darkness_score(frame_bgr):
29     gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
30     h_spread = histogram_spread(gray)
31     p_dark = pct_dark_pixels(gray)
32     b_mean = mean_brightness_score(gray)
33     e_norm = normalized_entropy_score(gray)
34     Q = W1 * h_spread + W2 * p_dark + W3 * b_mean + W4 * e_norm
35     return Q
36
37 def zero_dce_pp(frame_bgr, model):
38     """Pixel-wise curve enhancement (DCE-Net inference); model loaded
    upstream."""
39     return model.infer(frame_bgr)
40
41 def bilateral_denoise(frame_bgr, d=9, sigma_color=75, sigma_space=75):
42     return cv2.bilateralFilter(frame_bgr, d, sigma_color, sigma_space)

```

```
43
44 def wiener_deconvolution(frame_bgr, psf, K=0.01):
45     """Per-channel Wiener deconvolution in the frequency domain (Eq. 9)."""
46
47     out = np.zeros_like(frame_bgr, dtype=np.float32)
48     psf_fft = np.fft.fft2(psf, s=frame_bgr.shape[:2])
49     psf_conj = np.conj(psf_fft)
50     denom = (np.abs(psf_fft) ** 2) + K
51     for c in range(3):
52         G = np.fft.fft2(frame_bgr[:, :, c])
53         F_hat = (psf_conj / denom) * G
54         out[:, :, c] = np.real(np.fft.ifft2(F_hat))
55     return np.clip(out, 0, 255).astype(np.uint8)
56
57 def preprocess_frame(frame_bgr, zero_dce_model, psf):
58     Q = darkness_score(frame_bgr)
59     if Q >= THRESHOLD:                                     # "if (score < threshold):
60         image is dark?" -> yes
61         enhanced = zero_dce_pp(frame_bgr, zero_dce_model)
62         denoised = bilateral_denoise(enhanced)
63         deblurred = wiener_deconvolution(denoised, psf)
64     else:                                                   # no -> skip enhancement
65         branch
66         deblurred = wiener_deconvolution(frame_bgr, psf)
67     return deblurred    # ready for YOLO anomaly detection
```

Listing 1: Pre-anomaly-detection preprocessing pipeline

3 Stage 2: Post-Anomaly-Detection Pipeline

3.1 Pipeline Diagram

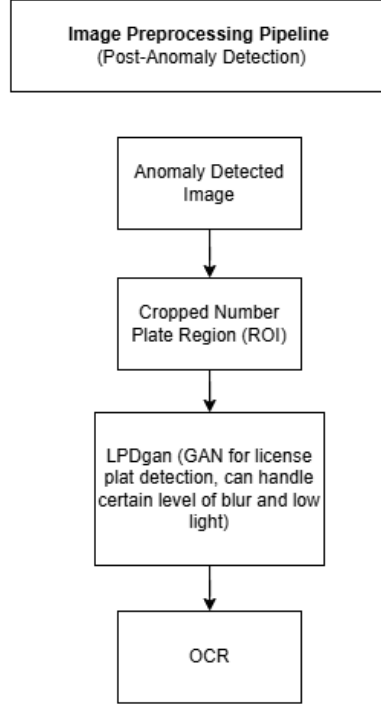


Figure 2: Post-Anomaly Image Pre-Processing

3.2 Stage Description

3.2.1 Region-of-Interest (ROI) Cropping

Once Stage 1 (fused with the violation-classification logic) flags a vehicle as an anomaly (any of 7 identified anomalies) its bounding box from the YOLO detector is used to localize and crop the **number-plate region**:

$$I_{ROI} = I_{enh}[y_0 : y_1, x_0 : x_1] \quad (12)$$

where (x_0, y_0, x_1, y_1) is the plate sub-region within (or derived from) the vehicle's bounding box, typically obtained from a dedicated plate-localization head or a fixed geometric prior relative to the vehicle box. Cropping restricts all subsequent, more expensive restoration computation to a small region rather than the full frame.

3.2.2 Plate Restoration – LPDGAN

I_{ROI} is frequently still degraded – by residual motion blur (vehicle was moving at the violation instant) and/or low light – even after Stage 1's frame-level correction, since frame-level deblurring parameters are tuned for the whole scene rather than the small, fast-moving plate region.

LPDGAN (License-Plate-Deblurring GAN) is a conditional generative-adversarial network specialized for this restoration sub-task. Its generator $\mathcal{G}$ produces a restored plate crop, while a discriminator $\mathcal{D}$ is trained adversarially to distinguish restored crops from real sharp plates, using the standard conditional-GAN minimax objective:

$$\min_{\mathcal{G}} \max_{\mathcal{D}} \mathbb{E}_{I_{sharp}} [\log \mathcal{D}(I_{sharp})] + \mathbb{E}_{I_{ROI}} [\log(1 - \mathcal{D}(\mathcal{G}(I_{ROI})))] \quad (13)$$

augmented at training time with a pixel-level reconstruction term (e.g. L_1 loss against the ground-truth sharp plate) to keep the restored output faithful to the actual characters rather than merely "plausible-looking":

$$\mathcal{L}_{total} = \mathcal{L}_{GAN} + \lambda \cdot \|\mathcal{G}(I_{ROI}) - I_{sharp}\|_1 \quad (14)$$

LPDGAN is chosen at this stage specifically because it can **handle a certain level of residual blur and low light** that survives the upstream Stage 1 corrections – i.e., it acts as a second, targeted restoration pass rather than a duplicate of the frame-level enhancement.

3.3 Algorithmic Summary – Stage 2

```

1 import cv2
2
3 def crop_plate_roi(frame, plate_bbox):
4     """plate_bbox: (x0, y0, x1, y1) from the plate-localization head."""
5     x0, y0, x1, y1 = plate_bbox
6     return frame[y0:y1, x0:x1]
7
8 def lpdgan_restore(plate_crop, generator_model):
9     """
10    Single forward pass through the LPDGAN generator G.
11    Handles moderate residual blur / low-light left after Stage 1.
12    """
13    restored = generator_model.infer(plate_crop)
14    return restored
15
16 def run_ocr(restored_plate, ocr_engine):
17     return ocr_engine.recognize(restored_plate)
18
19 def postprocess_anomaly(frame, plate_bbox, generator_model, ocr_engine):
20     roi = crop_plate_roi(frame, plate_bbox)
21     restored = lpdgan_restore(roi, generator_model)
22     plate_str = run_ocr(restored, ocr_engine)
23     return plate_str

```

Listing 2: Post-anomaly-detection plate restoration and OCR pipeline

4 End-to-End Pipeline

The two stages are intentionally decoupled: Stage 1 runs on *every* frame to support reliable detection, whereas Stage 2 is invoked *only* for the comparatively rare subset of vehicles classified as violations – keeping the expensive GAN-based restoration off the real-time critical path.

5 Important Logic Points

- **Quality-gated enhancement.** Zero-DCE++ is invoked conditionally, not unconditionally – only frames whose cumulative darkness score Q meets or exceeds θ enter the enhancement branch (Eq. 6). This avoids over-processing already well-lit frames.
- **Weighted, multi-statistic darkness score.** No single statistic (brightness alone, say) is trusted; Q combines histogram spread, dark-pixel fraction, mean brightness, and entropy, weighted by $w_1, \dots, w_4$ (Eq. 1), making the gate robust to corner cases (e.g. a bright headlight glare against an otherwise dark scene).
- **Deblurring is unconditional.** Unlike the brightness branch, Wiener deconvolution (Eq. 11) is applied to *every* frame regardless of the darkness decision, since motion blur is caused by vehicle/camera dynamics, not illumination.
- **Denoising follows enhancement, not before it.** Bilateral filtering is applied *after* Zero-DCE++ specifically because curve-based brightening amplifies sensor noise that was previously masked in dark regions.
- **Stage 2 is conditionally triggered.** The plate-restoration sub-pipeline (ROI crop $\rightarrow$ LPDGAN $\rightarrow$ OCR) only executes for frames/vehicles that the anomaly logic has already flagged as violations, keeping per-frame computation low for the (majority) non-violating traffic.
- **LPDGAN complements, not duplicates, Stage 1.** Stage 1 corrects the *whole frame*; LPDGAN performs a second, targeted restoration pass on the small plate crop, tolerating residual blur/low light that frame-level correction could not fully resolve.
- **Modularity.** Each block (Zero-DCE++, bilateral filter, Wiener deconvolution, YOLO, LPDGAN, OCR) is independently swappable, allowing the pipeline to be re-tuned or re-trained block-by-block as better models become available.

6 Summary

The Image Preprocessing Pipeline applies a two-stage, condition-aware enhancement strategy: a quality-gated, frame-level correction (darkness scoring $\rightarrow$ optional Zero-DCE++ $\rightarrow$ bilateral denoise $\rightarrow$ mandatory Wiener deblur) before anomaly detection, and a targeted, GAN-based plate restoration (LPDGAN) after a violation is flagged, immediately prior to OCR. This division of labor keeps real-time detection efficient while still providing high-fidelity restoration exactly where it is needed most – the license-plate region of a confirmed violator.

References

- [1] R. C. Gonzalez and R. E. Woods, <https://www.c172.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf>, 4th edition, Pearson, 2018.
- [2] Github reference for Zero-DCE <https://github.com/Li-Chongyi/Zero-DCE>
- [3] C. Guo et al., “Zero-DCE++: Efficient Deep Curve Estimation for Low-Light Image Enhancement,” <https://arxiv.org/pdf/2001.06826>, vol. 44, no. 10, pp. 7285–7298, 2022.
- [4] Github refernce for LPDgan – <https://github.com/haoyGONG/LPDGAN>